Initial question: "What is CSS specificity and how does it affect styling?"

Analysis: This question evaluates understanding of CSS specificity. It assesses the ability to explain how specificity is calculated using selector types and how it resolves style conflicts by prioritizing rules with higher specificity. The question also tests skills in managing complex style hierarchies and staying current with evolving CSS standards and best practices.

Answer: CSS specificity is a set of rules that determines which CSS styles are applied to an element when there are conflicting style declarations. The main specificity rules are:

- Inline styles have the highest specificity (1000 points).
- ID selectors have the next highest specificity (100 points).
- Class selectors, attribute selectors, and pseudo-classes have medium specificity (10 points each).
- Element selectors and pseudo-elements have the lowest specificity (1 point each).

When multiple selectors target the same element, the browser calculates the specificity score for each selector and applies the styles from the selector with the highest score. If selectors have equal specificity, the last declared rule takes precedence.

Here are some additional rules:

- The universal selector (`*`) has no specificity value (0 points).
- Combinators (like `>`, `+`, `~`) do not add to specificity.
- The `:is()` and `:not()` pseudo-classes do not add specificity on their own; instead, they adopt the highest specificity among their arguments.
- The specificity of `:has()` is determined by the most specific selector in its argument list, similar to `:is()` and `:not()`. However, when using relative selectors (like `>`, `+`, `~`) inside `:has()`, only the specificity of the most specific argument is considered, not the combinator itself.
- The `:where(z)` pseudo-class always has its specificity set to zero, regardless of its contents.
- Cascade layers (`@layer`) are applied in declared order, with later layers